

Informe de elaboracion del aplicativo GoToGym Developers

Proyecto: GoToGym Developers

Fecha de elaboracion del informe: 22 de junio de 2026

Periodo evidenciado de desarrollo: 25 de marzo de 2026 al 16 de junio de 2026

Repositorio analizado: gotogymdevelopers

Proposito del documento: presentar ante la universidad un informe tecnico y funcional detallado sobre la elaboracion de la pagina/aplicativo GoToGym Developers.

1. Resumen ejecutivo

GoToGym Developers es un aplicativo web orientado a la visualizacion, integracion y administracion de informacion relacionada con bienestar, salud digital, actividad fisica, dispositivos smartwatch, integraciones externas, datos de usuario y analitica corporativa. El proyecto evoluciono desde una estructura inicial de backend y validacion de usuario hasta convertirse en una consola web con login, control de roles, dashboard, cards dinamicas, modulos de usuario final, modulos para empresa/gimnasio, consumo de APIs protegidas con JWT y pruebas automatizadas.

El sistema fue construido principalmente con React + TypeScript + Vite en el frontend y Node.js + Express + TypeScript en el backend. La arquitectura se organiza por capas: rutas, controladores, servicios, repositorios, modelos, datos mock, hooks, componentes visuales, paginas y pruebas. Esta separacion permite mantener el codigo escalable, probar endpoints de forma independiente y conectar progresivamente con APIs reales de GoToGym.

Durante el desarrollo se identifican 80 commits en Git, distribuidos entre el 25 de marzo y el 16 de junio de 2026. La actividad se concentra en once dias de desarrollo registrados, con hitos claros: estructura base, API inicial, frontend SaaS, autenticacion, rutas protegidas, integracion con Azure Static Web Apps, smartwatch, cards, APP GOTO GYM, bienestar corporativo, miembros empresariales y RBAC.

2. Objetivo general

Desarrollar una plataforma web para GoToGym Developers que permita consultar, visualizar y administrar informacion de bienestar digital mediante una consola moderna, segura, modular y conectada a servicios backend, diferenciando la experiencia segun el tipo de usuario: usuario final, gimnasio/empresa y administrador.

3. Objetivos especificos

- Crear una interfaz web responsiva con home, login, dashboard, cards, smartwatch, APP GOTO GYM, bienestar corporativo y equipo empresa.
- Implementar autenticacion con JWT y control de acceso por roles/permisos.
- Consumir APIs internas y externas para informacion de usuario, bienestar, empresa, integraciones, bodygraph y smartwatch.
- Organizar el backend en rutas, controladores, servicios, repositorios y middlewares.
- Implementar estados de carga, error, fallback y datos mock para soportar pruebas y desarrollo incremental.
- Validar comportamiento critico mediante pruebas automatizadas de backend y frontend.
- Preparar el proyecto para despliegue web con configuracion compatible con Azure Static Web Apps y backend Node/Express.

4. Fechas de elaboracion y bitacora de desarrollo

La evidencia de Git muestra el siguiente avance del proyecto:

| Fecha | Commits | Actividad principal evidenciada |

| 2026-03-25 | 7 | Estructura inicial, endpoint REST de validacion de usuario, DTOs, pruebas unitarias, OpenAPI y configuracion base. |

| 2026-03-27 | 11 | Reinicio/orden del repositorio, base React + TypeScript, componente principal, README por carpetas, backend inicial y pruebas. |

| 2026-04-08 | 11 | Creacion del frontend, dashboard SaaS, header, sidebar, integraciones, acciones, resultados, notificaciones, backend Express y despliegue inicial. |

| 2026-04-21 | 9 | Rutas protegidas, home/login/dashboard, logout, JWT, smartwatch, cards interactivas, sidebar por rol y workflow de Azure Static Web Apps. |

| 2026-04-23 | 12 | Ajustes de cards, navegacion, leyenda de sueno, zona horaria Colombia, correcciones de despliegue y sincronizacion de historial. |

| 2026-05-12 | 3 | Seccion APP GOTO GYM con cards dinamicas, autenticacion JWT y despliegue a produccion. |

| 2026-05-27 | 9 | Developer console actualizada, bienestar con JWT real, mapeo de endpoints, cards de scores, despliegues y documentacion. |

| 2026-05-28 | 2 | Widgets de bienestar en dashboard y normalizacion del indice de felicidad. |

| 2026-06-02 | 6 | Home interactivo, modulos de bienestar de usuario, dashboard, smartwatch, login rediseñado y proxy backend corporativo. |

| 2026-06-09 | 8 | Seguridad de sesiones, dashboard corporativo, diseno de bienestar, endpoint corporativo, miembros empresariales y progress bars. |

| 2026-06-16 | 2 | RBAC, contexto tenant-aware y despliegue publico del frontend desde preproduccion. |

5. Estimacion de horas trabajadas

La estimacion se realiza a partir de la cantidad de commits, el periodo de desarrollo, la complejidad funcional y los modulos implementados. No equivale a un registro exacto de reloj, sino a una aproximacion razonable para informe academico.

| Area de trabajo | Horas estimadas |

| Planeacion, organizacion inicial, estructura de carpetas y documentacion base | 18 a 24 h |

| Backend Express/TypeScript, rutas, controladores, servicios, repositorios y middlewares | 42 a 56 h |

| Autenticacion, JWT, sesiones, RBAC, scopes, permisos y tenant context | 28 a 38 h |

| Frontend React/TypeScript: home, login, router, layout, header, sidebar y dashboard | 48 a 64 h |

| Modulos smartwatch, bodygraph, integraciones, cards y notificaciones | 40 a 54 h |

| APP GOTO GYM, consumo de coach context, extraccion de scores y visualizacion dinamica | 24 a 34 h |

| Bienestar corporativo, empresa, miembros, politicas agregadas y proxy API | 34 a 48 h |

| Pruebas automatizadas, correcciones TypeScript, QA y ajustes de despliegue | 26 a 36 h |

| Integracion con Azure Static Web Apps, configuracion de entorno y revisiones de produccion | 18 a 28 h |

| Ajustes visuales, responsividad, pulido de UI y estabilizacion | 28 a 40 h |

Estimacion total: entre 306 y 422 horas.

Estimacion media recomendada para presentacion: aproximadamente 360 horas de trabajo.

Esta cifra contempla investigacion, implementacion, correccion de errores, pruebas, despliegues, documentacion y ajustes visuales. La existencia de commits de limpieza del repositorio y despliegue se interpreta como trabajo de mantenimiento, no como incremento directo de funcionalidad.

6. Arquitectura general

El aplicativo se divide en dos grandes capas:

Frontend: aplicacion web desarrollada con React, TypeScript y Vite. Contiene paginas, componentes reutilizables, hooks, servicios HTTP, estilos, mocks y pruebas con Jest/Testing Library.

Backend: API desarrollada con Node.js, Express y TypeScript. Contiene rutas REST, controladores, servicios, middlewares de autenticacion/autorizacion, modelos, repositorios, datos mock y pruebas con Jest/Supertest.

6.1 Estructura principal del proyecto

- frontend/: codigo de la interfaz web, paginas, router, componentes, hooks, estilos, mocks y pruebas.
- backend/: API Express, rutas, controladores, servicios, repositorios, modelos, middlewares y tests.
- docs/: documentacion tecnica y reportes.
- scripts/: automatizaciones de documentacion, historial y generacion de informes.
- .github/: configuraciones de flujos de despliegue e integracion.
- appdesplegada-inspect/: carpeta de inspeccion y referencias de aplicacion desplegada, documentacion, assets y evidencia adicional.

7. Tecnologias utilizadas

| Capa | Tecnologia | Uso |

Frontend	React 18	Construccion de interfaces por componentes.
Frontend	TypeScript	Tipado estatico y contratos de datos.
Frontend	Vite	Servidor de desarrollo y build rapido.
Frontend	React Router DOM	Navegacion entre home, login y modulos protegidos.
Frontend	Axios / Fetch	Consumo de APIs y envio de token Bearer.
Frontend	Recharts	Visualizacion de graficas y metricas.
Backend	Node.js	Entorno de ejecucion JavaScript/TypeScript.
Backend	Express	Definicion de API REST.
Backend	jsonwebtoken	Emision y verificacion de tokens JWT.
Backend	cors	Control de origenes permitidos.
Pruebas	Jest	Pruebas unitarias y de integracion.
Pruebas	Supertest	Validacion HTTP del backend.
Despliegue	Azure Static Web Apps / App Service	Publicacion de frontend y backend/API.

8. Frontend del aplicativo

El frontend esta organizado para separar pantallas, componentes visuales, componentes de layout, servicios HTTP y hooks. La aplicacion se inicia desde frontend/index.tsx y frontend/App.tsx, y la navegacion principal se define en frontend/router/AppRouter.tsx.

8.1 Home

La pagina Home presenta la marca GoToGym Developers, historia, mision, enfoque de investigacion, contacto y enlaces sociales. Su objetivo es dar contexto institucional antes del ingreso a la consola. Incluye:

- Barra de navegacion superior.
- Logo de GoToGym.
- Hero con mensaje de bienestar inteligente.
- Tabs de contenido para historia, mision y Quantum Research.
- Seccion de soporte/contacto.
- Enlaces externos a redes sociales.

8.2 Login

La pantalla de login permite ingresar email y password. Consume loginWithApi, funcion que intenta autenticar contra /api/v1/auth/login y, si no existe ese endpoint, contempla fallback hacia /api/login/. El login:

- Valida que email y password no esten vacios.
- Muestra estado de envio.
- Presenta errores si credenciales o backend fallan.
- Guarda sesion con token, rol, permisos, tenant y refresh token si aplica.
- Redirige a /dashboard cuando la autenticacion es correcta.

8.3 Router y rutas protegidas

El router define rutas publicas y privadas:

- /: Home.
- /login: Login.
- /dashboard: dashboard protegido.
- /smartwatch: modulo de usuario final.
- /app-gotogym: modulo APP GOTO GYM.
- /business-wellbeing: modulo empresa/gimnasio/admin.
- /business-members: miembros empresariales.
- /cards: cards de usuario.

El componente ProtectedRoute valida autenticacion y roles permitidos. Si no hay sesion, redirige a login; si el rol no esta autorizado, redirige al dashboard.

8.4 Dashboard

El dashboard resume el estado general del sistema. Presenta metricas de integraciones activas, fuentes conectadas, ultima sincronizacion, eventos procesados, estado del sistema y actividad reciente. Se apoya en useDeveloperDashboard, hook que consulta integraciones, permite seleccionar fuentes y maneja estados de sincronizacion.

8.5 Header y sidebar

El layout de la consola incluye:

- Header con breadcrumb, descripcion de seccion, version, boton de cerrar sesion, campana de notificaciones y configuracion.

- Sidebar con navegacion dinamica segun rol.
- Secciones visibles por rol:
- Usuario final: Dashboard, Smartwatch, APP GOTO GYM y, cuando aplica, Equipo Empresa.
- Gym/empresa: Dashboard, Bienestar Corporativo y Equipo Empresa.
- Admin: acceso completo a secciones principales y modulos administrativos.

8.6 Cards

Las cards se usan para mostrar informacion sintetica y accionable. Existen cards para dashboard, smartwatch, APP GOTO GYM, bienestar empresarial, miembros y estado de integraciones. En APP GOTO GYM, las cards muestran scores de variables como actividad fisica, sueno, estres, intensidad, estabilidad emocional, vida social, HRV, edad biologica, calidad del sueno y ritmo circadiano.

8.7 Smartwatch

El modulo smartwatch consulta `/api/v1/smartwatch/metrics` con token Bearer. Si la API no responde, usa datos mock controlados para mantener la interfaz funcional. Esto permite desarrollo y demostracion aun cuando el backend real no este disponible. El modulo contempla:

- Metricas de salud y actividad.
- Estado de carga.
- Estado de error.
- Indicacion de fuente de datos: API o mock.

8.8 APP GOTO GYM

Este modulo consume informacion de `/api/coach_context/` mediante `wellbeingService`. El frontend extrae scores desde diferentes estructuras posibles: `if_snapshot`, `latest_record` y `wellbeing_experience_value_v1.if_variable_payload`. Esta decision hace robusto el consumo de APIs cuando el backend evoluciona.

Variables visualizadas:

- Nivel de actividad fisica.
- Horas de sueno promedio.
- Manejo del estres.
- Intensidad de entrenamientos.
- Estabilidad emocional.
- Vida social y conexiones.
- Variabilidad cardiaca.
- Edad biologica.
- Calidad del sueno.
- Sincronizacion del ritmo circadiano.

8.9 Bienestar corporativo

El modulo empresarial consulta bienestar agregado por ventana de dias. Permite analizar:

- Indice corporativo.
- Activacion.
- Recurrencia.
- Pausas completadas.

- Estado del programa.
- Pausas activas SG-SST.
- Variables IF agregadas.
- Mapa de riesgos.
- Fortalezas.
- Prioridades.
- Experiencias.
- Acciones recomendadas.
- Insights.

La interfaz evita mostrar datos sensibles individuales y favorece una lectura agregada de empresa.

8.10 Equipo empresa

El modulo Equipo Empresa presenta directorio, operacion y bienestar agregado por niveles. Incluye busqueda, filtro por estado, indicadores de miembros activos, onboarding, planes asignados y bienestar agregado. El diseno declara explicitamente que el detalle individual de salud requiere consentimiento, scopes y auditoria.

9. Backend del aplicativo

El backend esta construido con Express y TypeScript. La aplicacion principal se configura en backend/src/app.ts, donde se habilita CORS, JSON, rutas REST y middleware centralizado de errores.

9.1 Rutas principales

| Ruta | Metodo | Proposito | Seguridad |

| /api/v1/auth/login | POST | Autenticacion de usuario y emision de JWT. | Publica con credenciales. |

| /api/v1/auth/me | GET | Consultar usuario autenticado. | Bearer token. |

| /api/v1/smartwatch/metrics | GET | Obtener metricas smartwatch. | Bearer + scope smartwatch:read:self. |

| /api/v1/business/wellbeing/corporate | GET | Proxy de bienestar corporativo. | Bearer + rol empresa/admin + scope corporativo. |

| /api/integrations | GET | Listar integraciones. | Bearer + scope de lectura organizacional. |

| /api/integrations/:id/sync | POST | Sincronizar integracion. | Bearer + scope de sincronizacion. |

| /api/bodygraph/:integrationId | GET | Consultar datos BodyGraph por integracion. | Bearer + scope bodygraph:read:self. |

9.2 Autenticacion y autorizacion

El backend utiliza JWT con jsonwebtoken. Los tokens contienen:

- Identificador del usuario.
- Email.
- Rol.
- Scopes/permisos.
- Tenant.
- Organizacion activa.

- Tipo de token.

Los middlewares implementados son:

- requireAuth: valida encabezado Authorization Bearer y verifica token.
- requireRole: restringe por rol.
- requireScope: restringe por permiso/scope.
- requirePermission: alias semantico de scope.
- requireOrganizationAccess: valida que el usuario consulte una organizacion perteneciente a su tenant, salvo administradores.

9.3 Roles y permisos

El proyecto maneja una capa de RBAC en frontend y una capa de roles/scopes en backend. En frontend se reconocen roles user, gym y admin; en backend existen roles mas orientados a dominio como end_user, company_owner, company_manager, gotogym_admin e integrator.

Permisos destacados:

- smartwatch:read:self
- bodygraph:read:self
- coach_context:read:self
- corporate_wellbeing:read:organization
- integrations:read:organization
- integrations:sync:organization
- organization_members:read:organization
- applications:manage:organization
- billing:manage:organization

10. Consumo de APIs

El aplicativo consume APIs mediante Axios y Fetch. El servicio api.ts centraliza una instancia Axios con base URL configurable y agrega automaticamente el header Authorization: Bearer <token> cuando existe sesion.

10.1 API de usuario

La autenticacion se realiza por /api/v1/auth/login. El frontend guarda la sesion en sessionStorage, no en almacenamiento persistente, lo que reduce exposicion si el navegador se cierra. La funcion /api/v1/auth/me permite validar la sesion y obtener el usuario autenticado.

10.2 API de empresa

El modulo corporativo consume /api/v1/business/wellbeing/corporate, endpoint backend que funciona como proxy hacia https://api.gotogym.store/api/business/wellbeing/corporate/. El backend conserva el token Bearer y agrega parametros como org y days. La ventana days se normaliza entre 7 y 180 dias para evitar consultas fuera de rango.

10.3 API de coach context

El frontend consulta /api/coach_context/ con include_text=true y opcionalmente username. El payload puede contener perfil, documentos, dispositivos, snapshot IF, negocio y valor de experiencia de bienestar.

10.4 API de smartwatch

El frontend y backend usan `/api/v1/smartwatch/metrics`. El endpoint exige JWT y scope. El frontend implementa fallback mock para que la UI se mantenga operativa en ambiente de desarrollo.

10.5 API de integraciones y BodyGraph

Las integraciones se listan desde `/api/integrations` y pueden sincronizarse mediante POST `/api/integrations/:id/sync`. BodyGraph se consulta por integracion en `/api/bodygraph/:integrationId`, devolviendo informacion como ritmo cardiaco, pasos, sueno, estres, fuente y timestamp.

11. Seguridad

Medidas implementadas:

- Tokens JWT firmados con secreto configurable.
- Requerimiento de JWT_SECRET en produccion.
- Validacion de issuer.
- Validacion de expiracion en frontend.
- Uso de Authorization Bearer.
- Limpieza de sesion ante 401/403.
- Sesion en sessionStorage.
- Restriccion CORS a `https://developers.gotogym.store` y `http://localhost:5173`.
- Control por roles y scopes.
- Validacion de tenant/organizacion.
- Manejo centralizado de errores.
- Fallbacks controlados para desarrollo sin exponer datos reales.

12. Calidad y pruebas

El proyecto incluye pruebas automatizadas relevantes:

12.1 Backend

Pruebas con Jest y Supertest para:

- Login exitoso con JWT.
- Login invalido con 401.
- Consulta de usuario autenticado.
- Listado de integraciones.
- Sincronizacion de integraciones.
- Error 404 en integracion inexistente.
- Rechazo por permisos insuficientes.
- Consulta de BodyGraph.
- Consulta de metricas smartwatch.
- Requerimiento de Authorization Bearer.
- Rechazo de roles sin permisos.
- Proxy de bienestar corporativo.
- Normalizacion de dias entre 7 y 180.
- Rechazo de usuarios finales en endpoint corporativo.

12.2 Frontend

Pruebas para:

- RBAC y helpers de permisos.
- Persistencia y limpieza de sesion.
- Acceso de admin a modulos.
- Restriccion de usuarios y gym a modulos administrativos.
- Extraccion de scores de APP GOTO GYM.
- Hook de metricas smartwatch.
- Renderizado general de la aplicacion.

13. Despliegue y ambientes

El proyecto contiene configuraciones para despliegue de frontend con Azure Static Web Apps, configuracion staticwebapp.config.json, workflows y scripts de apoyo. El backend esta preparado para compilar TypeScript y ejecutarse desde dist/server.js.

Comandos principales:

- Frontend desarrollo: npm run dev dentro de frontend/.
- Frontend build: npm run build dentro de frontend/.
- Backend desarrollo: npm run dev dentro de backend/.
- Backend build: npm run build dentro de backend/.
- Backend tests: npm test dentro de backend/.
- Frontend tests: npm test dentro de frontend/.

14. Modulos funcionales entregados

| Modulo | Estado | Descripcion |

| Home | Implementado | Presentacion institucional, historia, mision, investigacion y contacto. |

| Login | Implementado | Autenticacion con API, manejo de errores y redireccion. |

| Dashboard | Implementado | Vista general de integraciones, fuentes, eventos y actividad. |

| Permisos/RBAC | Implementado | Roles, permisos, sesiones, rutas protegidas y tenant context. |

| Smartwatch | Implementado | Consumo de metricas con fallback mock. |

| APP GOTO GYM | Implementado | Cards dinamicas de bienestar personal desde coach context. |

| Cards | Implementado | Visualizacion de indicadores y scores clave. |

| Integraciones | Implementado | Listado, seleccion y sincronizacion de fuentes. |

| BodyGraph | Implementado | Datos corporales y de salud asociados a integraciones. |

| Bienestar corporativo | Implementado | Dashboard agregado para empresa/gimnasio/admin. |

| Equipo empresa | Implementado | Directorio, operacion y bienestar agregado. |

| Notificaciones | Implementado | Alertas asociadas a integraciones y eventos criticos. |

| Pruebas | Implementado | Jest, Supertest y Testing Library. |

15. Retos tecnicos abordados

- Mantener compatibilidad entre endpoints propios y endpoints heredados o externos.
- Manejar diferentes estructuras de payload para APP GOTO GYM sin romper la UI.
- Separar datos individuales sensibles de datos corporativos agregados.
- Implementar control de permisos en frontend y backend.
- Conservar una experiencia demostrable aunque una API real no este disponible.

- Normalizar errores HTTP y limpiar sesion ante accesos no autorizados.
- Preparar el aplicativo para ambientes locales y productivos mediante variables de entorno.
- Mejorar progresivamente la interfaz manteniendo componentes reutilizables.

16. Conclusiones

GoToGym Developers alcanzo una version funcional de consola web con arquitectura moderna, separacion frontend/backend, autenticacion, permisos, consumo de APIs y modulos diferenciados para usuario y empresa. El aplicativo no se limita a una pagina estatica: integra datos, valida sesiones, protege rutas, transforma respuestas de API y presenta informacion de bienestar mediante componentes visuales.

El proyecto evidencia un proceso incremental de desarrollo entre marzo y junio de 2026, con avances tecnicos, pruebas, despliegues, documentacion y refinamiento visual. La estimacion media de trabajo es de aproximadamente 360 horas, sustentada en el volumen de commits, la variedad de modulos, la cantidad de integraciones y la complejidad de seguridad/autorizacion.

Como trabajo universitario, el aplicativo demuestra competencias en desarrollo full stack, arquitectura por capas, consumo de servicios REST, manejo de autenticacion JWT, diseno de interfaces, pruebas automatizadas, documentacion tecnica y preparacion para despliegue en la nube.